

10 — Python imports, type annotations, and parsing / Python 导入、类型标注与解析

The question / 问题

What do S1 imports, from __future__ import annotations, and parse mean in the WO Table 4 script?

WO Table 4 脚本中的 S1 导入、from __future__ import annotations 和 parse 分别是什么意思？

This guide uses Code_Python_rogp39/Case2-WO-Table4-Reproduction/wo_eq36_fullscan_IK_casadi.py. The small examples explain Python behavior; they do not run the process model. The version-specific behavior below was checked with Python 3.9.25.

本文对应 Code_Python_rogp39/Case2-WO-Table4-Reproduction/wo_eq36_fullscan_IK_casadi.py。小例子只演示 Python 行为，不运行工艺模型。下面涉及版本的行为已用 Python 3.9.25 核对。

1. Type annotations describe expected data / 类型标注说明预期的数据类型

```
def square(x: float) -> float:
    return x * x
```

x: float describes the expected input type. -> float describes the expected return type. The calculation is still x * x. Python does not automatically enforce these hints or convert the data: square(3) returns the integer 9. See [Python type hints](#)^[1].

x: float 说明预期输入是浮点数；-> float 说明预期返回浮点数。实际计算仍是 x * x。Python 不会自动按标注检查或转换数据：square(3) 仍返回整数 9。参见 [Python 类型提示](#)^[1]。

In the WO script, read this parameter one piece at a time:

把 WO 脚本中的这个参数拆开看：

```
p: dict[str, float] | None = None
```

Part 部分	Meaning 含义
p	Parameter name 参数名
dict[str, float]	Dictionary with string keys and float values, such as {"rho": 50.0} 键是字符串、值是浮点数的字典，如 {"rho": 50.0}
None	The type hint also allows None 类型说明也允许 None
= None	If the caller omits p, its default value is None 调用时不提供 p，默认值就是 None

A dictionary can contain zero, one, or many key–value pairs. In `dict[str, float]`, `str` describes the keys and `float` describes the values; the two type names do not mean there is only one pair. For example:

字典（dictionary）可以包含零个、一个或多个键值对（key–value pairs）。 `dict[str, float]` 中的 `str` 说明键的类型，`float` 说明值的类型；这两个类型名称并不是说“只能放一对”。例如：

```
p = {"rho": 50.0, "k": 0.2, "volume": 60.0}
print(len(p))      # 3
print(p["rho"])    # 50.0
print(p["k"])      # 0.2
```

These are illustrative entries, not a complete validated WO parameter set. Each key is a string and each value here is a float. The annotation does not prescribe particular key names, their count, or physical units; the function body may impose additional requirements. `{}` is an empty dictionary, while `None` means no dictionary was supplied. They are different values.

这些条目只是教学示例，并不是完整、已验证的 WO 参数集。每个键都是字符串，每个值在这里都是浮点数。这个标注没有规定必须有哪些键、键值对数量或物理单位；函数内部代码可能另有要求。`{}` 是空字典，而 `None` 表示没有提供字典，两者不是同一个值。

Likewise, `sc: list[dict[str, Any]]` describes a list of scenario dictionaries. `Any` leaves value types unrestricted; a record can contain float flows and integer indices. `-> np.ndarray` describes an array return value.

同理，`sc: list[dict[str, Any]]` 表示由情景字典组成的列表。`Any` 不限定值的类型，因此一条记录可以同时包含浮点流量和整数编号。`-> np.ndarray` 说明预期返回数组。

2. Why the future import is needed here / 这里为什么要用 future import

2.1 A number and a piece of text are different / 2.1 数字和文字不是一回事

Start with these two lines. The only difference is the quotation marks:

先看下面两行，区别只是有没有引号：

```
number = 1 + 2
text = "1 + 2"

print(number)      # 3
print(type(number)) # <class 'int'>
print(text)        # 1 + 2
print(type(text))  # <class 'str'>
```

For `number`, Python performs the addition and stores the result 3. For `text`, Python stores the characters `1 + 2` as a string. **Saving those characters does not perform the addition, now or automatically later.**

`number` 那一行让 Python 做加法，保存结果 3。`text` 那一行只保存 `1 + 2` 这几个字符，类型是字符串（string）。**保存这些字符，不会做加法，也不会过一会儿就自动做加法。**

A separate instruction can ask Python to execute the saved text as an expression. Here `eval()` does that for our fixed arithmetic example:

如果另外下达指令，Python 才会把保存的文字当作表达式执行。下面的 `eval()` 就是对这个固定的加法例子提出这样的要求：

```
text = "1 + 2"
result = eval(text)
print(result) # 3
print(text)   # 1 + 2
```

result receives 3; text itself remains the string "1 + 2". This example explains the difference between storing text and asking Python to execute it. You do not need to add `eval()` to the WO script.

result 得到 3，而 text 本身仍然是字符串 "1 + 2"。这个例子只是帮助区分“保存文字”和“要求 Python 执行文字中的表达式”。你的 WO 脚本不需要因此添加 `eval()`。

2.2 Apply that distinction to the parameter annotation / 2.2 再看参数旁边的类型说明

In `p: dict[str, float] | None = None`, the part after the colon and before `= None` describes the expected type of `p`: **a dictionary with string keys and float values, or None**. The `|` means “or” here. The final `= None` separately specifies the default value if no argument is supplied.

在 `p: dict[str, float] | None = None` 中，冒号后面、`= None` 前面的部分，是对 `p` 的类型说明：**字符串键、浮点数值**的字典，**或者 None**。这里的 `|` 表示“或者”；最后的 `= None` 则单独规定“不传参数时默认用什么值”。

Without the `future` import, Python 3.9 tries to turn this annotation into a type description it can work with when defining the function. That requires support for `|` between these types. Python 3.9 does not have that support, so the definition raises `TypeError`. This use of `|` was introduced in [Python 3.10](#) ^[2].

不加 `future` import 时，Python 3.9 在定义函数时，就会尝试把这段标注变成它能使用的类型信息。这需要支持“用 `|` 把类型组合起来”的功能。Python 3.9 没有这个功能，所以函数定义会报 `TypeError`。这种 `|` 用法是在 [Python 3.10](#) ^[2] 中加入的。

With `from __future__ import annotations`, Python 3.9 instead keeps this annotation as the string `"dict[str, float] | None"`. Just as saving `"1 + 2"` does not require addition, saving this text does not require performing the unsupported `|` operation. The function can therefore be defined and called. See [PEP 563](#) ^[3].

加上 `from __future__ import annotations` 后，Python 3.9 改为把这段标注保存成字符串 `"dict[str, float] | None"`。就像保存 `"1 + 2"` 不需要做加法，保存这份文字也不需要执行它不支持的 `|` 运算。因此函数可以正常定义和调用。参见 [PEP 563](#) ^[3]。

Run the following as a complete, separate Python 3.9 script:

将下面内容作为一个完整、独立的 Python 3.9 脚本运行：

```
from __future__ import annotations

def identity(p: dict[str, float] | None = None):
    return p

parameters = {"rho": 50.0, "k": 0.2}
result = identity(parameters)
print(result)                # {'rho': 50.0, 'k': 0.2}
print(type(result).__name__)  # dict

annotation = identity.__annotations__["p"]
print(annotation)             # dict[str, float] | None
print(type(annotation).__name__) # str
```

There are two separate things to inspect: `result` is the real dictionary returned by the function; `annotation` is the saved text describing the parameter. `identity.__annotations__["p"]` simply retrieves that description. **The dictionary has not become a string.**

这里分开看两样东西：`result` 是函数返回的真实字典；`annotation` 是保存下来的参数类型说明。

`identity.__annotations__["p"]` 只是取出这份说明的写法。字典没有变成字符串。

2.3 Is it equivalent to a comment? / 2.3 这是不是相当于注释掉了?

It resembles a comment in that it does not immediately perform the `|` operation, but it is not discarded. A normal `#` comment is not stored as the function's annotation; this description is stored in `__annotations__` and can be retrieved by tools. The future import also does not add automatic checking of the argument values.

可以把它理解为“暂时不执行里面的 `|`”，但它并没有被丢弃。普通 `#` 注释不会作为函数的类型标注保存；这份说明则保存在 `__annotations__` 中，工具仍能取出来查看。`future import` 也不会自动检查你传入的数据类型。

2.4 When can an error still occur? (Optional) / 2.4 那什么时候还会报错? (可先跳过)

An ordinary `identity(parameters)` call uses the dictionary and runs `return p`; it does not ask Python to convert the saved annotation text into type information. But `typing.get_type_hints(identity)` makes that additional request. On Python 3.9, it then encounters the unsupported `|` operation and raises `TypeError`. This is a separate action, like explicitly calling `eval(text)` in the arithmetic example. **A stored string does not execute itself, and the future import does not upgrade Python.**

普通的 `identity(parameters)` 调用拿到字典，然后执行 `return p`，不会要求 Python 把保存的类型文字转换为类型信息。但 `typing.get_type_hints(identity)` 会额外提出这个要求。在 Python 3.9 中，这时又遇到不支持的 `|` 运算，才会报 `TypeError`。这是额外的一步，就像前面的例子里特意调用了 `eval(text)`。**字符串不会自己执行，future import 也没有升级 Python。**

If a Python 3.9 program needs that extra tool, the compatible spelling below expresses the same intended type. This is an optional alternative; the original WO script is not being changed.

如果 Python 3.9 程序确实需要那个额外工具，可以使用下面兼容的写法，表达相同的类型要求。这只是可选的替代方式，没有修改原 WO 脚本。

```
from typing import Optional

def identity(p: Optional[dict[str, float]] = None):
    return p
```

`Optional[dict[str, float]]` also means “this dictionary type or None.” Neither spelling restricts the dictionary to a single key–value pair.

`Optional[dict[str, float]]` 同样表示“这种字典或者 None”。两种写法都不限制字典只能有一个键值对。

`__future__` is a special standard-library module. Python recognizes `from __future__ import annotations` as an instruction to enable this annotation behavior in the file. *Dunder* means *double underscore*: `__future__` has two underscores at each end. Put the statement before ordinary imports; a module docstring and comments may precede it. See [future statements](#) ^[4].

`__future__` 是特殊的标准库模块。整句 `from __future__ import annotations` 相当于“在本文件开启上述标注处理方式”。dunder 是 double underscore (双下划线) 的简称, `__future__` 前后各有两个下划线。这句放在普通导入之前; 前面可以有文件说明字符串和注释。参见 [future 语句](#) ^[4]。

3. Read the S1 imports by their job / 按用途理解 S1 导入

S1 makes tools available. These import lines do not start the WO optimization.

S1 准备后面要用的工具。这些导入语句还没有开始 WO 优化计算。

Import 导入语句	Use in this script 本脚本中的用途
<code>import argparse</code>	Read options such as <code>--output-dir</code> 读取 <code>--output-dir</code> 等运行选项
<code>import csv</code>	Read historical MATLAB results and write result/comparison tables 读取 MATLAB 历史结果, 写出结果表和比较表
<code>from datetime import datetime, timezone</code>	Record UTC dates and create timestamped run names 记录 UTC 日期时间, 生成带时间戳的运行名称
<code>import hashlib</code>	Record SHA-256 file fingerprints for source tracking 记录文件的 SHA-256“指纹”, 用于追溯来源
<code>import json</code>	Save configuration, versions, and run records 保存配置、版本和运行记录
<code>from pathlib import Path</code>	Locate files, build paths, and create output directories 定位文件、拼接路径、创建输出目录
<code>import platform</code>	Record the Python version 记录 Python 版本
<code>import time</code>	Measure solver elapsed time with <code>perf_counter()</code> 用 <code>perf_counter()</code> 计算求解耗时
<code>from typing import Any</code>	Supply Any for type annotations 提供类型标注中的 Any
<code>import casadi as ca</code>	Build symbolic variables, objectives, and constraints; call IPOPT 建立符号变量、目标和约束, 调用 IPOPT
<code>import numpy as np</code>	Work with numerical arrays, initial guesses, and results 处理数值数组、初值和结果

`import csv` gives names such as `csv.DictWriter`. `from pathlib import Path` allows `Path(...)` directly. `as np` gives NumPy a short name. These are three common import forms.

`import csv` 后用 `csv.DictWriter`; `from pathlib import Path` 后直接用 `Path(...)`; `as np` 给 NumPy 起简称。这是三种常见的导入形式。

```
V = ca.MX.sym("V") # Symbolic decision variable
v0 = np.array([60.0]) # Numerical initial value
```

These two lines require the CasADi and NumPy imports above. CasADi represents the optimization model; NumPy handles numerical data. Later, `ca.nlpsol(..., "ipopt", ...)` selects IPOPT as the solver.

这两行需要先导入上表中的 CasADi 和 NumPy。v 是符号决策变量，v0 是含有具体数值的初值数组。CasADi 表达优化模型，NumPy 处理数值数据；后面的 `ca.nlpsol(..., "ipopt", ...)` 指定 IPOPT 求解器。

4. Parse means read according to rules / Parse 就是按规则解析

When parsing Python text such as `a = 1 + 2`, a parser identifies an assignment and an addition. Executing it calculates 3 and assigns it to `a`.

解析 Python 文本 `a = 1 + 2` 时，解析器识别出“赋值”和“加法”的结构。执行时才算出 3，并把它赋给 `a`。

In this script, `parser.parse_args()` instead reads command-line options and stores their values in `args`. The options `--output-dir` and `--historical-csv` become `args.output_dir` and `args.historical_csv`; omitted options are `None`. Here `type=Path` actually converts a supplied value to a path object; it is not a type annotation. See [argparse](#)^[5].

在本脚本里，`parser.parse_args()` 读取命令行选项，并把值放进 `args`。`--output-dir` 和 `--historical-csv` 分别对应 `args.output_dir` 和 `args.historical_csv`；未提供时为 `None`。这里的 `type=Path` 会实际把传入的值转换成路径对象，它不是类型标注。参见 [argparse](#)^[5]。

Run the [small standalone example](#) from the handbook root. It requires Python 3.9+ and no external packages; it creates no output directory or simulation results.

在手册根目录运行这个独立小例子。需要 Python 3.9+，无需额外安装库；它不会创建输出目录或生成模拟结果。

```
python examples/10_python_annotations_and_parsing.py --output-dir results/demo
```

Expected output:

预期输出：

```
p value: {'rho': 50.0}
p type: dict
annotation: dict[str, float] | None
annotation type: str
output path: results/demo
```

The dictionary is unchanged, its type annotation is stored as text, and the command-line path has been read. The real WO script then passes that path to `run_sweep()` to begin its calculations.

字典保持原样，类型标注保存为文字，命令行路径也已读出。实际 WO 脚本随后把路径传给 `run_sweep()`，才进入计算流程。

参考资料 (References)

1. <https://docs.python.org/3.9/library/typing.html>
2. <https://docs.python.org/3.10/whatsnew/3.10.html#pep-604-new-type-union-operator>

3. <https://peps.python.org/pep-0563/>
4. https://docs.python.org/3.9/reference/simple_stmts.html#future-statements
5. <https://docs.python.org/3.9/library/argparse.html>